

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 2

Programación Dinámica

30 de octubre de 2025

Juan Ignacio Brugger
103583
Carolina Amarilla
111995

Federico Codino
103533
Rodolfo Albornoz
107975

Índice

1. Análisis del problema	4
1.1. Ecuación de recurrencia	5
1.2. Algoritmo propuesto	5
2. Descripción del algoritmo	6
2.1. Algoritmo	6
2.1.1. Función principal	6
2.1.2. Función cargar_data()	7
2.1.3. Función calcular_ataque()	7
2.1.4. Función reconstruir_estrategia()	8
2.1.5. Función imprimir_resultados()	8
2.2. Análisis de complejidad	9
2.3. Impacto de la variabilidad de los valores x_k y $f(j)$	9
3. Mediciones de tiempos	9
4. Conclusión	10
5. Correcciones	11
5.1. Ecuación de recurrencia	11
5.2. Análisis de Optimalidad	11

Trabajo Práctico 2: Programación Dinámica para el Reino de la Tierra

Consigna

1. Hacer un análisis del problema, plantear la ecuación de recurrencia correspondiente y proponer un algoritmo por programación dinámica que obtenga la solución óptima al problema planteado: Dada la secuencia de llegadas de enemigos $x_1, x_2, \dots, x_n$ y la función de recarga $f(\cdot)$ (dada por una tabla, con lo cual puede considerarse directamente como una secuencia de valores), determinar la cantidad máxima de enemigos que se pueden atacar, y en qué momentos se harían los correspondientes ataques.
2. Escribir el algoritmo planteado. Describir y justificar la complejidad de dicho algoritmo. Analizar si (y cómo) afecta a los tiempos del algoritmo planteado la variabilidad de los valores de las llegadas de enemigos y recargas.
3. Analizar si (y cómo) afecta a la optimalidad del algoritmo planteado la variabilidad de los valores de las llegadas de enemigos y recargas.
4. Realizar ejemplos de ejecución para encontrar soluciones y corroborar lo encontrado. Adicionalmente, el curso proveerá con algunos casos particulares que deben cumplirse su optimalidad también.
5. De las pruebas anteriores, hacer también mediciones de tiempos para corroborar la complejidad teórica indicada. Realizar gráficos correspondientes. Generar todo set de datos necesarios para estas pruebas.
6. Agregar cualquier conclusión que parezca relevante.

1. Análisis del problema

Para abordar la implementación de este trabajo, primero decidimos describir los elementos que componen el problema y cómo se relacionan entre sí.

- **Secuencia de llegadas enemigas:** Tenemos una secuencia de soldados que llegan minuto a minuto, dada por $x_1, x_2, \dots, x_n$, donde x_i representa la cantidad de enemigos que aparecen en el minuto i .
- **Ataques del equipo:** El equipo puede realizar ataques que eliminan enemigos. La fuerza de cada ataque depende del tiempo que se ha acumulado energía desde el último ataque. Para esto contamos con una función $f(j)$, que indica la cantidad de soldados que se pueden eliminar si han pasado j minutos desde el último ataque.
- **Decisión en cada minuto:** En cada minuto se debe decidir si atacar o esperar.
 - *Atacar:* Se eliminan $\min(x_i, f(j))$ enemigos y la energía acumulada se reinicia para el próximo minuto.
 - *Esperar:* No se eliminan enemigos, pero la energía acumulada aumenta un minuto, lo que permite realizar un ataque más fuerte en el futuro.
- **Relación entre los elementos:** Los elementos interactúan minuto a minuto. En cada instante llegan x_i enemigos y se debe decidir si atacar o no. Lo que puede provocar la eliminación de los enemigos, reiniciando la energía acumulada o aumentando la fuerza para el próximo ataque.
- **Objetivo del problema:** Determinar la cantidad máxima de enemigos que se pueden eliminar y en qué minutos conviene realizar los ataques para lograr ese máximo.
- **Ejemplo:** Supongamos que tenemos 4 minutos y los datos son:

$$x = [5, 2, 6, 3] \quad f = [1, 3, 4, 6]$$

- **Minuto 1:** Energía = 1, enemigos = 5.
 - Atacar: se eliminan $\min(5, 1) = 1$ enemigo.
 - Esperar: se acumula energía para el minuto 2.
- **Minuto 2:** Según la decisión anterior, energía = 1 (si atacamos en el minuto 1) o 2 (si esperamos). Enemigos = 2.
 - Con energía = 1, se eliminan $\min(2, 1) = 1$ enemigo.
 - Con energía = 3, se eliminan $\min(2, 3) = 2$ enemigos.
 - Esperar: se acumula energía para el minuto 3
- **Minuto 3:** Energía = 1, 3 o 4 según decisiones previas. Enemigos = 6.
 - Con energía = 1, se eliminan $\min(6, 1) = 1$ enemigo.
 - Con energía = 3, se eliminan $\min(6, 3) = 3$ enemigos.
 - Con energía = 4, se eliminan $\min(6, 4) = 4$ enemigos.
 - Esperar: se acumula energía para el minuto 4
- **Minuto 4:** Energía = 1, 2, 3 o 4 según la combinación de decisiones anteriores. Enemigos = 3.
 - Con energía = 1, se eliminan $\min(3, 1) = 1$ enemigo.
 - Con energía = 3, se eliminan $\min(3, 3) = 3$ enemigos.
 - Con energía = 4, se eliminan $\min(3, 4) = 3$ enemigos.
 - Con energía = 6, se eliminan $\min(3, 6) = 3$ enemigos.

Para resolver este problema, utilizaremos Programación Dinámica. Una técnica que nos permite explorar de forma implícita el espacio de posibles soluciones. La idea es descomponer el problema en subproblemas más pequeños que luego se combinan para construir soluciones de problemas más grandes. En este caso, cada minuto representa un subproblema: decidir si atacar o esperar. La decisión tomada afecta los minutos siguientes, ya que cambia la energía disponible y la cantidad de enemigos que se pueden eliminar. Con la memorización de subproblemas anteriores, evitamos recalcular escenarios repetidos y reducimos la complejidad del problema, logrando encontrar la cantidad máxima de enemigos eliminados.

1.1. Ecuación de recurrencia

Suponiendo que conocemos las soluciones de los subproblemas más pequeños, definimos $OPT(k)$ como la cantidad máxima de enemigos que se pueden eliminar considerando los primeros k minutos.

Cada minuto k puede o no ser un minuto de ataque:

- **No atacar en el minuto k :** En este caso, la solución óptima para los primeros k minutos es la misma que para los primeros $k - 1$ minutos:

$$OPT(k - 1)$$

- **Atacar en el minuto k :** Entonces debemos considerar el último minuto previo en que se atacó, digamos p (con $p = 0$ si no hubo ataques antes). La cantidad de enemigos eliminados en el minuto k será $\min(x_k, f(k - p))$, y se suma al valor óptimo hasta el minuto p :

$$OPT(p) + \min(x_k, f(k - p))$$

De estas dos opciones debemos seleccionar siempre la que maximice el total de enemigos eliminados. Por lo tanto, la ecuación de recurrencia queda:

$$OPT(k) = \max \left(OPT(k - 1), \max_{0 \leq p < k} (OPT(p) + \min(x_k, f(k - p))) \right)$$

donde $OPT(0) = 0$, y p representa el minuto del último ataque anterior a k . La primera opción corresponde a *no atacar* en k , mientras que la segunda corresponde a *atacar* en k .

1.2. Algoritmo propuesto

Para resolver el problema de manera óptima utilizamos programación dinámica. La idea general del algoritmo es la siguiente:

1. Inicialización:

- Creamos un registro para almacenar, para cada minuto, la máxima cantidad de enemigos que se pueden eliminar considerando los minutos anteriores.
- Creamos otro registro para guardar, para cada minuto, la decisión óptima: cuántos minutos atrás fue el último ataque.

2. Procesamiento minuto a minuto:

- Para cada minuto k , evaluamos todas las posibles duraciones desde el último ataque hasta este minuto.
- Para cada posible duración, calculamos cuántos enemigos se eliminarían si atacamos ahora sumando lo que se eliminó hasta el último ataque.
- Elegimos la duración que maximice el total de enemigos eliminados y registramos tanto la cantidad máxima como la decisión tomada.

3. Reconstrucción de la estrategia:

- Una vez calculadas todas las cantidades máximas y las decisiones, recorremos los minutos de atrás hacia adelante.
- Cada vez que encontramos un minuto marcado como ataque según la decisión óptima, lo registramos en la estrategia y retrocedemos hasta el último ataque.
- Los minutos que no están marcados como ataque se registran como “Cargar”, es decir, esperar para acumular energía.

Este enfoque garantiza que se explora todo el espacio de decisiones de manera eficiente, evitando recalcular subproblemas ya resueltos, y permite determinar la solución óptima al problema.

2. Descripción del algoritmo

2.1. Algoritmo

A continuación se muestra el código de todo el algoritmo.

```
1 def reconstruir_estrategia(max_eliminados, decision):
2     n = len(max_eliminados)
3     estrategia = ["Cargar"] * n
4     k = n - 1
5     while k >= 0:
6         estrategia[k] = "Atacar"
7         k = k - decision[k] - 1
8     return estrategia
9
10 def calcular_ataque(n, x, f):
11     max_eliminados = [float('-inf')] * n
12     decision = [-1] * n
13
14     for k in range(n):
15         for j in range(k + 1):
16             prev_idx = k - j - 1
17             prev_val = max_eliminados[prev_idx] if prev_idx >= 0 else 0
18             val = min(x[k], f[j]) + prev_val
19             if val > max_eliminados[k]:
20                 max_eliminados[k] = val
21                 decision[k] = j
22     return max_eliminados, decision
```

A continuación se realiza el análisis del algoritmo implementado en lenguaje Python, describiendo su comportamiento y calculando su complejidad.

2.1.1. Función principal

```
1 def main():
2     inicio = time.time()
3
4     ruta_archivo = sys.argv[1]
5
6     n, x, f = cargar_data(ruta_archivo)
7     max_eliminados, decision = calcular_ataque(n, x, f)
8     estrategia = reconstruir_estrategia(max_eliminados, decision)
9     imprimir_resultados(max_eliminados, estrategia)
10
11     fin = time.time()
12     print(f"Tiempo total de ejecucion: {fin - inicio:.6f} segundos")
```

La función se encarga de usar la ruta de archivo para llamar a *cargar_data()* que devuelve las variables para la función *calcular_ataque()*, que retorna la estrategia óptima y la cantidad de enemigos que esta elimina, con lo que procede a llamar a *reconstruir_estrategia()* y luego a la

función *imprimir_resultados()* que muestra por pantalla la mejor estrategia y a cuántos enemigos elimina dicha esta.

La complejidad de esta función será la que tenga mayor orden entre las funciones *cargar_data()*, *calcular_ataque()*, *reconstruir_estrategia()*, e *imprimir_resultados()* esto debido a que son llamadas de forma secuencial, sin ningún tipo de bucle y sin haber anidaciones entre ellas. Esta función será la que decida la complejidad de todo el código ya que al ser la principal, es la encargada de desencadenar la ejecución de todo el código.

2.1.2. Función *cargar_data()*

La función se encarga de leer la primera línea de código, que será un número n , para proceder a leer n líneas que guardará en una lista, que representará los soldados que aparecerán en cada momento y nuevamente volverá a leer n líneas que guardará en una lista y en este caso representa la energía resultante de acumularla esa cantidad de veces. Finalmente devolverá estas tres variables obtenidas.

```
1 '''
2 Recibe la ruta a un archivo recibida por linea de comando
3
4 Devuelve la cantidad de minutos a considerar (n), los xi (x) y los
5 valores que corresponden a la funcion f(.) (f)
6 '''
7 def cargar_data(filepath):
8     with open(filepath) as file:
9         filas = file.read().strip().splitlines()[1:]
10
11     n = int(filas[0])
12     x = list(map(int, filas[1:1 + n]))
13     f = list(map(int, filas[1 + n:1 + 2 * n]))
14
15     return n, x, f
```

Complejidad de la función *cargar_data()*

La complejidad de esta función depende de leer cada carácter del registro y recorrer todas las líneas, si bien lo separa en 3 partes, lee primero una línea, luego n líneas y luego vuelve a leer n líneas. Esto significa que prevalecerá la más grande siendo la de recorrer n líneas. Esto resulta en una función $O(n)$.

2.1.3. Función *calcular_ataque()*

La función evalúa todas las posibilidades en los n momentos, sobre acumular una cantidad de energía j y sumandola a un momento previamente evaluado que encaje dentro de los márgenes de tiempo, detectará cuál es el óptimo para ese tiempo y eventualmente llegara al óptimo del tiempo n . Devolverá dos listas, una con las decisiones óptimas para alcanzar el óptimo en n y otra con la máxima cantidad total de enemigos eliminados hasta ese momento,

```
1 '''
2 Recibe la cantidad de minutos a considerar (n), los xi (x) y los
3 valores que corresponden a la funcion f(.) (f)
4
5 Devuelve una lista con las decisiones optimas para alcanzar el ptimo en n, y otra
6 lista con la maxima cantidad total de enemigos eliminados al momento
7 '''
8 def calcular_ataque(n, x, f):
9     max_eliminados = [float('-inf')] * n
10     decision = [-1] * n
11
12     for k in range(n):
13         for j in range(k + 1):
14             prev_idx = k - j - 1
```

```
14     prev_val = max_eliminados[prev_idx] if prev_idx >= 0 else 0
15     val = min(x[k], f[j]) + prev_val
16     if val > max_eliminados[k]:
17         max_eliminados[k] = val
18         decision[k] = j
19     return max_eliminados, decision
```

Complejidad de la función calcular_ataque()

Esta función setea secuencialmente dos listas, lo que en ambos casos es $O(n)$, para luego implementar dos for anidados, donde solo hay instrucciones de tiempo constante, estos bucles dependen uno de n y el otro de la variable del for que lo anida haciendo que esto sea $O(n^2)$, ya que tiene complejidad mayor que el de inicializar las listas.

2.1.4. Función reconstruir_estrategia()

Crea una lista de tamaño n , llena de strings 'Cargar', luego la función recorre la lista de índices, donde se decidió atacar, producida por calcular_ataque(), y los usa para poner en la posición que indica los strings 'Atacar' en la primera lista. Finalmente devuelve esa lista que es la estrategia óptima.

```
1 '''
2 Recibe una lista con las decisiones optimas para alcanzar el optimo en n, y otra
3 lista con la maxima cantidad total de enemigos eliminados al momento
4 Devuelve una lista con la estrategia optima para resolver el problema (Con strings
5 'Cargar' y 'Atacar' segun corresponde con la estrategia obtenida)
6 '''
7 def reconstruir_estrategia(max_eliminados, decision):
8     n = len(max_eliminados)
9     estrategia = ["Cargar"] * n
10    k = n - 1
11    while k >= 0:
12        estrategia[k] = "Atacar"
13        k = k - decision[k] - 1
14    return estrategia
```

Complejidad de reconstruir_estrategia()

La complejidad de esta función es la de mayor orden entre crear una lista de tamaño n con elementos seteados, lo cual tiene complejidad $O(n)$ y la del bucle while que recorre el listado de decisiones, como en el peor de los casos se atacará en todos los turnos, la complejidad también es $O(n)$. Como ambas tienen complejidades iguales y no están anidadas, el resultado final es $O(n)$.

2.1.5. Función imprimir_resultados()

Esta función muestra por pantalla la estrategia óptima y la cantidad de soldados eliminados con esta.

```
1 '''
2 Recibe una lista con las decisiones ptimas para alcanzar el ptimo en n, y otra
3 lista con la maxima cantidad total de enemigos eliminados al momento
4 Imprime la estrategia para obtener la soluci n ptima y la cantidad m xima de
5 tropas que fueron eliminadas
6 '''
7 def imprimir_resultados(max_eliminados, estrategia):
8     print(f"Estrategia: {estrategia}")
9     print(f"Cantidad de tropas eliminadas: {max_eliminados[-1]}")
```


Complejidad de imprimir_resultados()

La complejidad de esta función es de $O(n)$ ya que muestra todos los n elementos de la lista y luego muestra una vez la cantidad de enemigos eliminados.

2.2. Análisis de complejidad

Como analizamos anteriormente, la complejidad de este algoritmo esta definida por su función `main`, que a su vez está definida por `cargar_data()`, `calcular_ataque()`, `reconstruir_estrategia()` e `imprimir_resultados()`. Con la información que tenemos podemos hacer el cálculo:

$$T(n) = O(n) + O(n^2) + O(n) + O(n)$$

Resultando que la complejidad del algoritmo es:

$$O(n^2)$$

Como podemos ver lo que determina la complejidad es el cálculo del ataque.

2.3. Impacto de la variabilidad de los valores x_k y $f(j)$

- **Tiempos del algoritmo:** La complejidad temporal del algoritmo sigue siendo $O(n^2)$, independientemente de los valores de las llegadas de enemigos x_k o de la función de recarga $f(j)$. Esto se debe a que el número de operaciones depende únicamente del tamaño de la secuencia n . La variabilidad de los valores solo afecta los cálculos internos de $\min(x_k, f(j))$, pero no altera la cantidad de iteraciones de los bucles anidados.
- **Optimalidad del algoritmo:** La optimalidad no se ve afectada por la variabilidad de los valores de las llegadas de enemigos x_k ni de la función de recarga $f(j)$. Esto se debe a que el algoritmo evalúa explícitamente todas las combinaciones posibles de decisiones "Cargar" o "Atacar" para cada minuto.

Independientemente de los valores específicos de x_k y $f(j)$, el algoritmo calcula la cantidad máxima de enemigos eliminados considerando todas las alternativas, garantizando que siempre se obtenga la solución óptima. Por lo tanto, cualquier cambio en las entradas afecta únicamente los resultados numéricos de la cantidad de enemigos eliminados en cada escenario, pero no compromete la optimalidad de la estrategia final.

3. Mediciones de tiempos

Con el objetivo de corroborar la complejidad del algoritmo, se realizaron mediciones utilizando conjuntos de datos de distintos tamaños. Los tamaños considerados fueron: 10, 50, 100, 200, 400, 600, 800, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500 y 5000 elementos.

Los valores de cada conjunto fueron generados de forma pseudoaleatoria utilizando el módulo `random` del lenguaje, con una semilla fija (`seed = 2698`) para asegurar la reproducibilidad de los resultados. Además, se aplicó un ajuste por cuadrados mínimos sobre las curvas correspondientes a las complejidades teóricas $O(n^2)$ y $O(n)$, con el fin de facilitar la comparación.

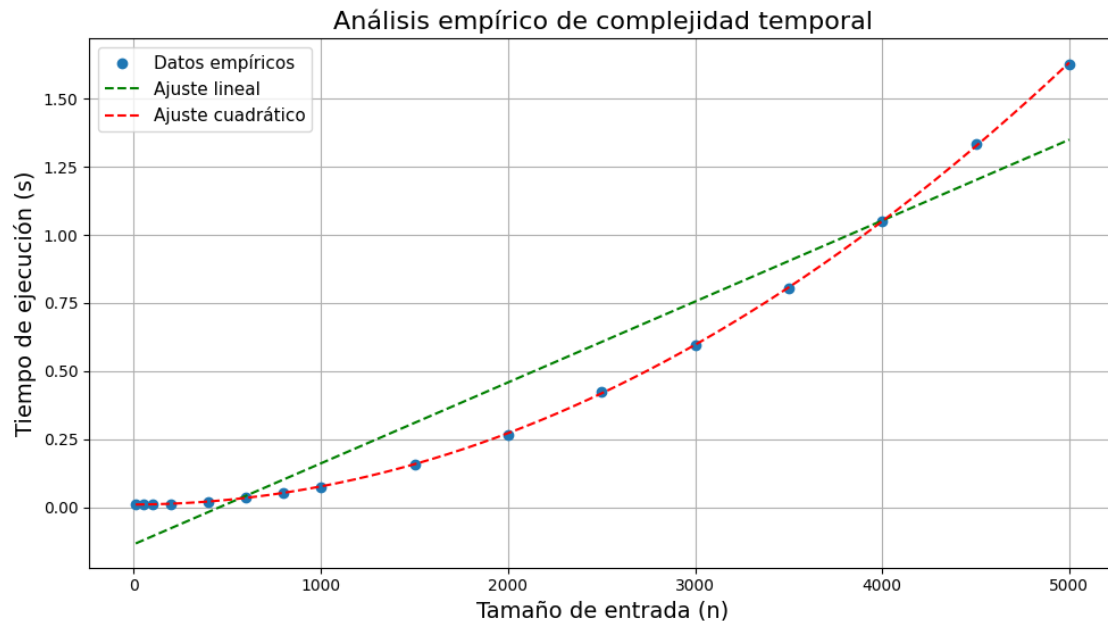


Figura 1: Comparación de los tiempos medidos con las curvas teóricas $O(n^2)$ y $O(n)$.

Como se observa en la figura, el algoritmo se ajusta mejor a la curva $O(n^2)$, lo que confirma que la complejidad obtenida en la práctica coincide con la complejidad teórica calculada en la sección anterior. Esto implica que el tiempo de ejecución del algoritmo es proporcional a n^2 , siendo n el tamaño de los datos de entrada.

4. Conclusión

En este trabajo se desarrolló un algoritmo de programación dinámica que permite determinar la estrategia óptima para maximizar la cantidad de enemigos eliminados, considerando la secuencia de llegadas y la función de recarga. Las pruebas realizadas con conjuntos de datos de tamaños variados (desde 10 hasta 5000 elementos) demostraron que el algoritmo siempre encuentra la cantidad máxima de enemigos que se pueden eliminar, garantizando la optimalidad de la solución en todos los casos evaluados.

Por ejemplo, en un conjunto de prueba con 5000 elementos, el algoritmo logró eliminar la totalidad de enemigos posibles según la función de recarga, mostrando consistencia en la toma de decisiones sobre cuándo atacar y cuándo esperar. La medición de tiempos mostró que el crecimiento del tiempo de ejecución se ajusta claramente a la curva $O(n^2)$, coincidiendo con la complejidad teórica calculada.

Asimismo, se verificó que la variabilidad en los valores de llegadas de enemigos x_k y la función de recarga $f(j)$ no afecta ni la optimalidad ni la eficiencia del algoritmo.

Si bien se intentó desarrollar un algoritmo con mejor complejidad, la naturaleza del problema y la necesidad de considerar todas las combinaciones posibles de decisiones nos llevó nuevamente a una solución de complejidad $O(n^2)$. A pesar de esto, el algoritmo diseñado logra de manera eficiente y correcta determinar la estrategia óptima y demuestra un comportamiento coherente con la teoría.

5. Correcciones

5.1. Ecuación de recurrencia

En la implementación, el caso de “no atacar” ya está incluido dentro del máximo general, por lo que no es necesario el máximo exterior sobre $OPT(k-1)$. La ecuación de recurrencia utilizada en el algoritmo queda entonces:

$$OPT(k) = \max_{0 \leq p < k} (OPT(p) + \min(x_k, f(k-p)))$$

donde $OPT(0) = 0$, y p representa el minuto del último ataque anterior a k .

5.2. Análisis de Optimalidad

Caso base

Para $k = 0$, solo podemos atacar en el primer minuto con 1 minuto de carga, por lo que la ecuación de recurrencia se reduce a:

$$OPT(0) = \min(x_0, f(1))$$

No hay combinaciones con ataques anteriores, por lo que claramente lo óptimo es atacar en ese primer minuto.

Caso $i < k$

En el caso donde hay un i que representa óptimos anteriores a k , el algoritmo explora todas las posibles combinaciones donde el último ataque antes del minuto que ocurrieron en algún i y se cargaron j minutos antes de atacar en k . Para cada combinación válida, suma:

$$OPT(k-j-1) + \min(x_k, f(j))$$

y toma el máximo entre ellas.

Como por inducción $OPT(k-j-1)$ es óptimo, y como el algoritmo toma el máximo de todas estas combinaciones, el resultado también será óptimo para $OPT(k)$